Conclusion

Conclusion I

`template_search_project` packages a complete, configurable, reproducible literature workflow into the Research Project Template's two-layer architecture. By keeping discovery, export, and synthesis in three orthogonal infrastructure modules, the project demonstrates that ambitious research automation can still respect the template's principles:

Conclusion I

- **Single source of truth** — `Paper` for discovery, `BibEntry` for export, structured `SynthesisResult` records for LLM output.
- **Test-driven development** — every module is covered by real-data tests; HTTP backends are exercised through `pytest-httpserver`, the LLM bridge through deterministic local callables.
- **Thin orchestrator pattern** — `scripts/run_search_pipeline.py` does only argument parsing, configuration loading, and I/O; all logic lives in `infrastructure/` or `src/`.
- **No mocks** — neither in the new infrastructure modules nor in the project test suite.
- **Multi-project support** — the project lives alongside `template_code_project/` and follows the same layout, so the existing pipeline runner discovers and executes it without modification.
- **Reproducibility** — deterministic search caching, on-disk enrichment caching, and pinned LLM seeds make a single `manuscript/config.yaml` the only artifact a reviewer needs.

Conclusion I

We close with three concrete extensions that build naturally on this foundation:

Conclusion I

1. **Crossref TDM full-text fetch** for non-arXiv DOIs, completing the abstract-to-fulltext picture without changing the project's API.
2. **CSL-JSON export** alongside BibTeX, enabling Zotero / Mendeley / Pandoc-CSL workflows from the same `BibDatabase`.
3. **Vector recall on `LocalBackend`** for sufficiently large curated corpora, gated behind an optional dependency and an explicit benchmark-derived crossover threshold.

Conclusion I

The infrastructure modules are deliberately small and stable; the project that exercises them is deliberately small and explicit. Together they show that *domain-specific research automation* and *template-strict architectural discipline* are compatible — and, in fact, mutually reinforcing.

The bundled `data/corpus.json` is a deterministic workflow fixture containing citation-shaped optimisation records (Boyd and Vandenberghe 2004; Nocedal and Wright 2006; Nesterov 2013; Kingma and Ba 2014; Reddi et al. 2018; Peng 2011). It ensures that the auto-generated `manuscript/references.bib` contains citation-ready entries that downstream tooling can resolve; it does not establish empirical literature findings.